

Spark Concepts — Questions & Answers

Week 6 Data Engineering Assessment — Celebal Technologies

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Think of a Spark application like a construction project. The **Driver** is the site manager — it's the process that runs your main code, builds the execution plan (the DAG), and decides how work should be split up. It never does the heavy lifting itself; it just plans and coordinates.

The **Cluster Manager** is like the staffing agency. Its job is purely to hand out resources — CPU cores and memory — to the application. Spark can work with different cluster managers: its own built-in Standalone manager, or external ones like YARN, Kubernetes, or Mesos. It doesn't know anything about your actual Spark code; it just allocates machines/containers on request.

The **Executors** are the actual construction workers. These are processes launched on worker nodes that execute the real tasks (reading data, filtering, aggregating, writing output) and hold data in memory/disk for caching. Each executor reports its progress and results back to the Driver, and the Driver assembles the final answer.

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

When you write a chain like `df.filter(...).select(...).groupBy(...)`, Spark doesn't run any of it immediately. It just records each step as a node in a logical plan. Only when you call an **action** — like `show()`, `count()`, or `write()` — does Spark actually execute anything.

This matters a lot for performance because it gives Spark's Catalyst optimizer a chance to look at the *entire* chain before running it, instead of executing each step one-by-one blindly. It can then:

- Push filters as early as possible (filter before you even read unnecessary columns)
- Skip computing columns or rows that are never actually used in the final output
- Combine several narrow operations into a single pass over the data instead of writing and re-reading intermediate results

In short: eager execution would process every intermediate step in full even if most of it turns out to be wasted work. Lazy evaluation lets Spark plan the shortest real path to your final answer before spending any cluster resources.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
df = spark.read.csv(
    "data/source.csv",
    header=True,
    inferSchema=True
```

)

`header=True` tells Spark the first line of the file is column names, not data. `inferSchema=True` makes Spark scan the data once to guess appropriate types (integer, double, string, etc.) instead of treating everything as a string.

Q4: What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

CSV is row-based: each line on disk holds one complete record, with every column's value written one after another. To read even a single column, Spark still has to scan through every row from start to finish.

Parquet is columnar: values from the same column are stored together in contiguous blocks. This has two big performance benefits:

- **Column pruning** — if your query only needs 2 out of 20 columns, Spark can read just those columns' data from disk and skip the rest entirely.
- **Better compression** — values in the same column tend to be similar to each other (same data type, often repeating values), so columnar data compresses far better than mixed row data.

Parquet also stores the schema and per-column statistics (min/max) in its metadata, which enables optimizations like predicate pushdown that a plain CSV simply cannot support.

Q5: Given a DataFrame `df`, write a query to select the columns `product_id` and `price` where the category is 'Electronics'.

```
result_df = df.filter(df.category == "Electronics") \
               .select("product_id", "price")

# or, using col()
from pyspark.sql.functions import col
result_df = df.filter(col("category") == "Electronics") \
               .select("product_id", "price")

result_df.show()
```

Q6: Write the code to "revise" a DataFrame by renaming the column `old_name` to `new_name` and casting the price column from a String to a Double.

```
from pyspark.sql.types import DoubleType

revised_df = df.withColumnRenamed("old_name", "new_name") \
               .withColumn("price", df["price"].cast(DoubleType()))
```

Q7: How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Every RDD/DataFrame in Spark keeps track of exactly how it was derived from its parent — which transformation was applied, and to which upstream partitions. Chained together across the whole job, this forms a **Directed Acyclic Graph (DAG)**, also called the lineage graph.

If a worker node crashes mid-job and some partitions are lost, Spark doesn't need a backup copy of that data sitting somewhere else. Instead, it looks at the lineage graph, figures out exactly which transformations produced the lost partitions, and simply **recomputes only those partitions** on a healthy executor — starting from the original source data or the nearest cached checkpoint.

This is what makes Spark resilient without the overhead of constantly replicating data the way a traditional distributed database would.

Q8: Write a query to filter a DataFrame `df_orders` for rows where the status is 'Completed' AND the amount is greater than 1000.

```
from pyspark.sql.functions import col

filtered_orders = df_orders.filter(
    (col("status") == "Completed") & (col("amount") > 1000)
)
filtered_orders.show()
```

Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Predicate Pushdown means Spark takes the filter condition in your query (the "predicate") and pushes it **down into the file-reading layer itself**, instead of reading the entire file into memory first and filtering afterward.

Parquet files store per-column, per-row-group statistics (like min and max values) in their metadata/footer. When Spark sees a filter such as `price > 1000`, it checks those statistics first. If a whole row group's maximum price is, say, 800, Spark knows in advance that group can't possibly contain a match — so it skips reading that block of data from disk entirely.

The practical effect is a big reduction in both disk I/O and memory usage, since only the row groups that could actually satisfy the condition are decompressed and loaded. This is something CSV cannot do at all, since it has no stored statistics and no way to skip sections of the file — every row has to be read and checked in application code.

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

```
from pyspark.sql.functions import col, round as spark_round

df = df.withColumn(
    "final_price",
    spark_round(col("base_price") * 1.18, 2)
)
```

Q11: What is the difference between Transformations and Actions? Provide two examples of each.

Transformations are lazy — they describe a step to be taken but don't run anything immediately. They always return a new DataFrame/RDD. Examples: `filter()`, `select()`, `withColumn()`, `groupBy()`, `join()`.

Actions are what actually trigger Spark to execute the whole chain of transformations built up so far, and they return a concrete result (not another DataFrame) — a number, a list of rows, printed output, or a file written to disk. Examples: `count()`, `show()`, `collect()`, `write()`.

A simple way to remember it: transformations build the plan, actions run the plan.

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where `user_id` is null, and save the result as a CSV at "path/to/output".

```
from pyspark.sql.functions import col

df = spark.read.parquet("path/to/input")

clean_df = df.filter(col("user_id").isNotNull())

clean_df.write.mode("overwrite") \
    .option("header", True) \
    .csv("path/to/output")
```

Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

The difference comes down to **where the Driver process runs**.

In **Client Mode**, the Driver runs on the machine that submitted the job — for example, your own laptop or an edge/gateway node — while the Executors run out on the cluster. This is convenient for interactive work like notebooks and debugging, since you get logs and output directly in your terminal. The downside is that if your local machine loses its network connection or shuts down, the whole application dies with it, since the Driver depends on that machine staying alive.

In **Cluster Mode**, the Driver itself is launched *inside* the cluster (on one of the worker nodes), managed by the Cluster Manager just like the Executors are. Once submitted, your local machine can disconnect entirely and the job keeps running. This is the standard choice for production batch jobs, since it doesn't depend on any single external machine staying online.

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
from pyspark.sql.functions import col

filtered_df = df.filter(
    (col("region") == "North") | (col("priority") == "High")
)
filtered_df.show()
```

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

`collect()` pulls **every single row** of the DataFrame from all the executors back to the Driver process, as a single in-memory Python/Scala collection. On a multi-terabyte dataset, that means trying to cram terabytes of data into the memory of one machine (the Driver) — which will almost certainly crash it with an out-of-memory error, and even if it didn't, it would be painfully slow and pointless for a human to look at millions of rows at once anyway.

`show(5)`, on the other hand, asks Spark to compute and return just a handful of rows (5 by default) for a quick visual sanity check. Spark is smart enough to only materialize as much data as needed to produce those rows, so memory usage on the Driver stays minimal regardless of how large the underlying dataset is.

The rule of thumb: use `show()`/`take()` for exploration and debugging, and reserve `collect()` only for cases where you're certain the final result set is small enough to fit comfortably in driver memory.